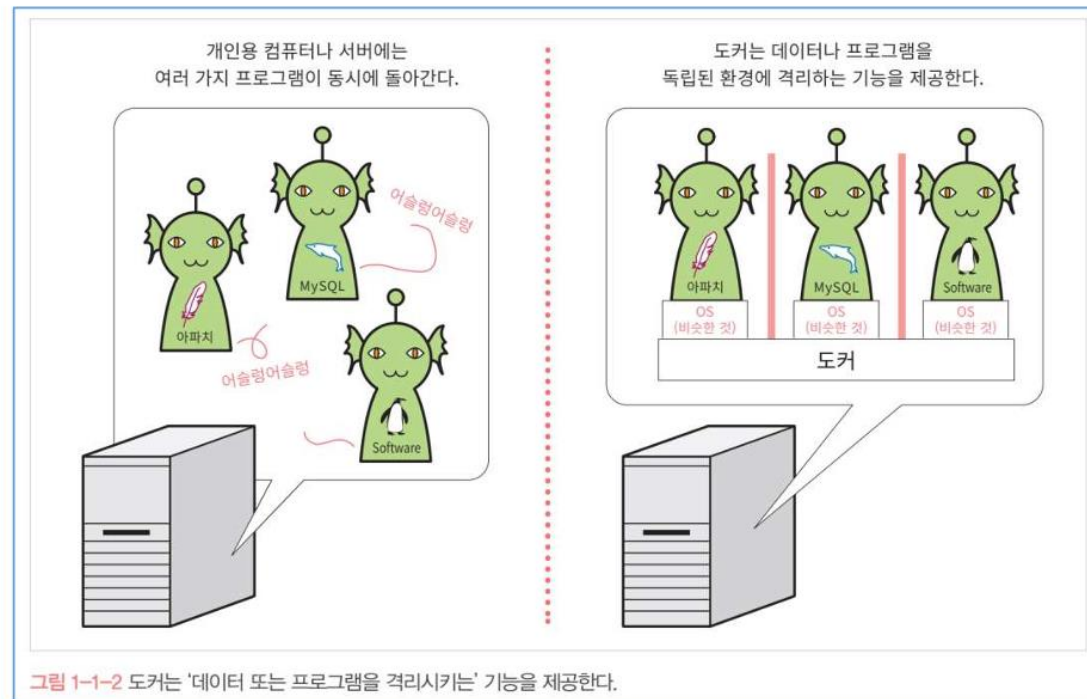


도커(Docker) 기초

• 도커(Docker)란?

- > 컨테이너를 활용하여 소프트웨어를 배포 및 구동하기 위한 오픈 소스 플랫폼
- > 데이터 또는 프로그램을 각각 독립된 환경에 격리시키는 기능을 제공한다.
- > 다른 컨테이너의 환경을 쉽게 그대로 재현할 수 있는 이미지를 제공한다.

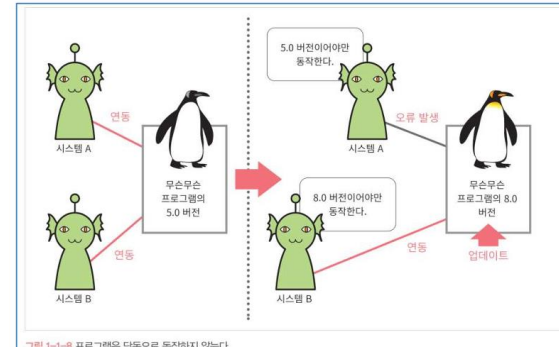
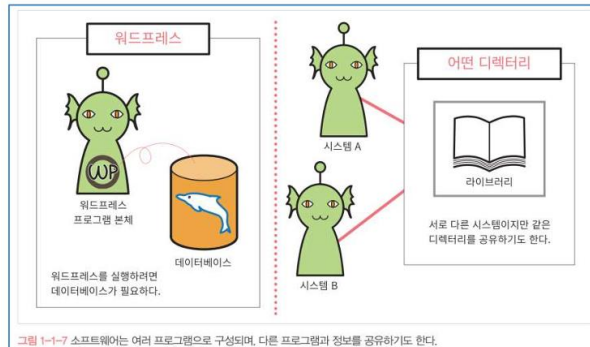
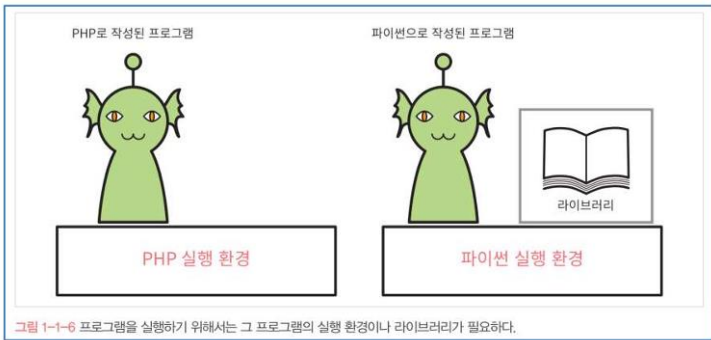


도커 기초

• 독립된 환경이 필요한 이유

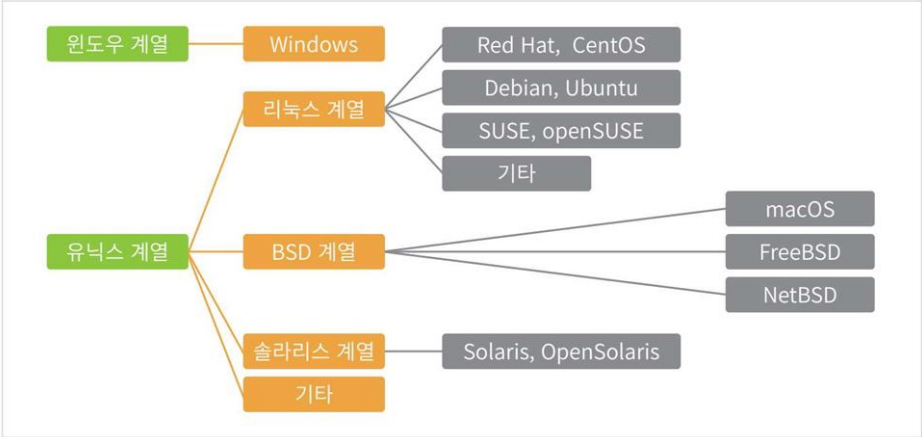
- > 프로그램을 실행하려면 특정 버전의 실행 환경·라이브러리가 함께 필요 (예: PHP 실행환경, 파이썬 라이브러리)
- > 워드프레스가 반드시 MySQL 데이터베이스를 필요로 하는 것처럼, 프로그램 사이에는 의존 관계가 존재
- > 서로 다른 프로그램이 같은 라이브러리를 공유하면, 한쪽을 업데이트할 때 다른 프로그램이 깨질 수 있음
- > 도커는 프로그램을 컨테이너 단위로 완전히 격리하여 이 문제를 근본적으로 해결

03



• 서버의 두 가지 의미

- > 기능적 의미: 어떤 서비스(Service)를 제공(Serve)하는 것
- > 물리적 의미: 그 서비스를 제공하는 컴퓨터 자체
- > 소프트웨어를 설치하면 그 기능을 하는 서버가 됨
 - 하나의 물리 서버에 여러 기능적 서버가 함께 존재할 수 있음
- > 그리고 서버의 운영체제는 주로 유닉스 계열인 리눅스를 사용함



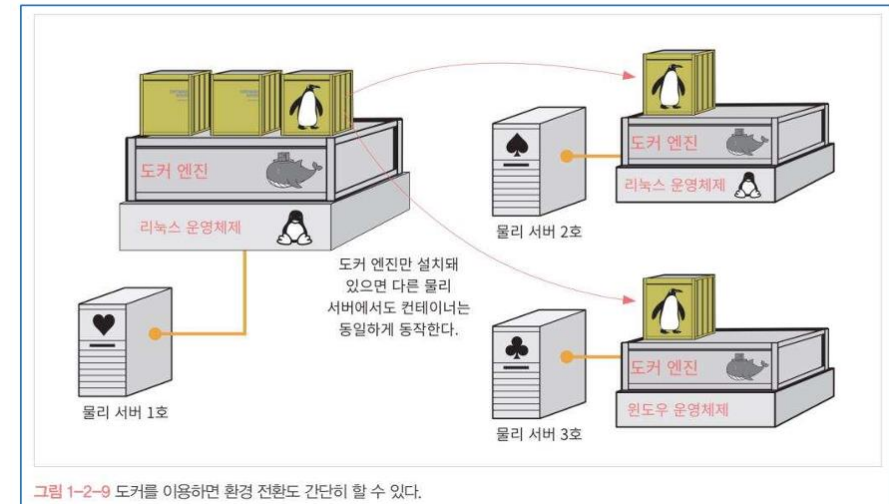
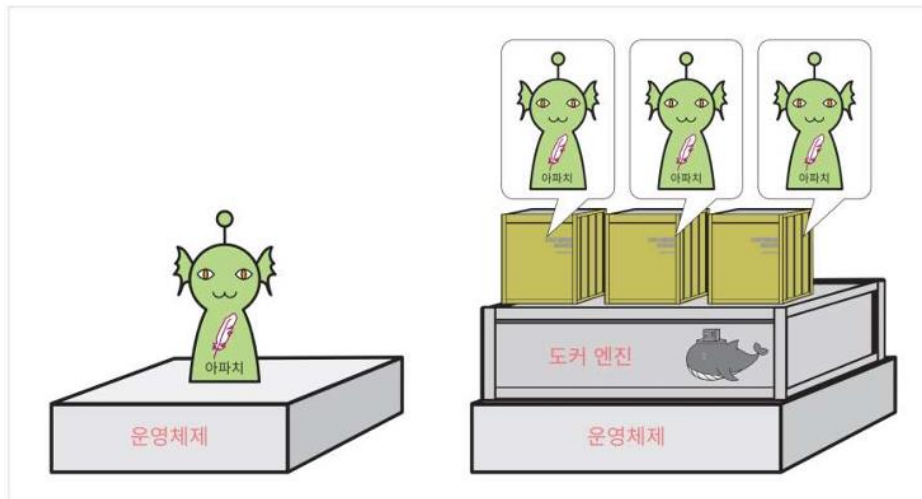
서버 종류	대표 소프트웨어
웹 서버	Apache, Nginx, IIS
메일 서버	Postfix, Dovecot
데이터베이스 서버	MySQL, PostgreSQL, MariaDB
파일 서버	Samba

도커 기초

• 도커로 여러 서버를 안전하게 운영하기

- > 컨테이너 단위로 격리되어, 하나의 물리 서버에서 여러 서비스를 동시에 안전하게 운영 가능
- > 물리 서버 수를 줄여 비용을 절감할 수 있음
- > 개발 환경을 운영 환경으로 그대로 옮길 수 있어 환경 차이로 인한 문제를 예방
- > 동일하게 구성한 컨테이너를 다른 물리 서버로 자유롭게 이동 가능(운영체제와 무관)

05

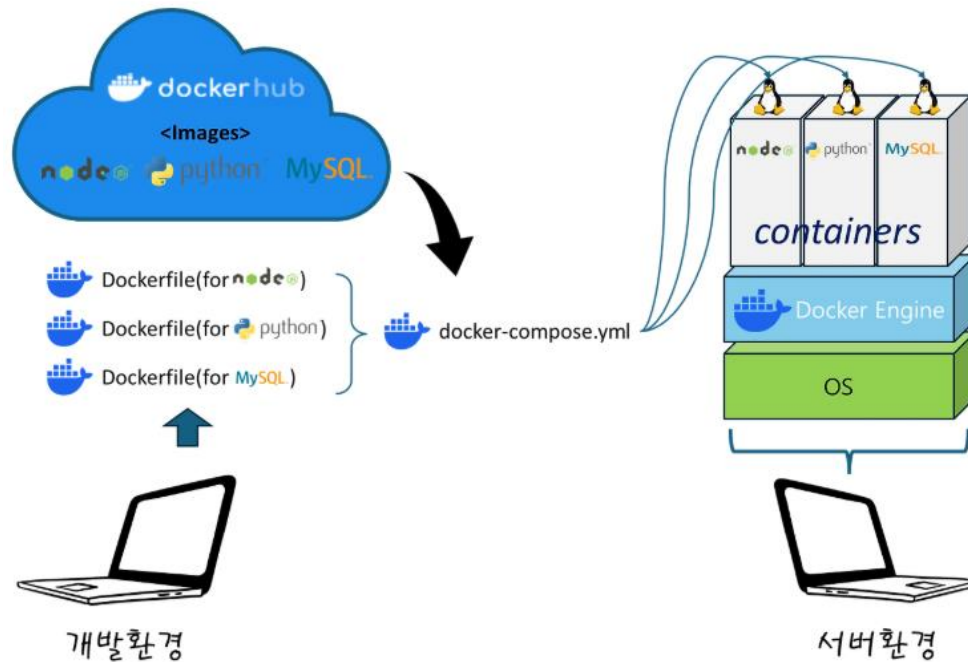


도커 기초

• 도커의 주요 활용 사례

- > 팀원 전체에게 동일한 개발 환경을 제공 (프로젝트별 컨테이너로 관리)
- > 새로운 OS·라이브러리 버전을 운영 반영 전에 안전하게 테스트
- > 동일한 서버를 여러 대 빠르게 확장(스케일 아웃)해야 할 때 — 웹 서버·API 서버에 특히 유용

06



도커 기초

• 도커의 구조 — 왜 가벼운가

- > 가상머신은 애플리케이션마다 커널을 포함한 Guest OS 전체를 하이퍼바이저 위에 올려야 해서 무거움
- > 도커 컨테이너는 커널을 직접 포함하지 않고 호스트 OS의 커널을 공유하기 때문에 훨씬 가볍고 빠르게 실행됨

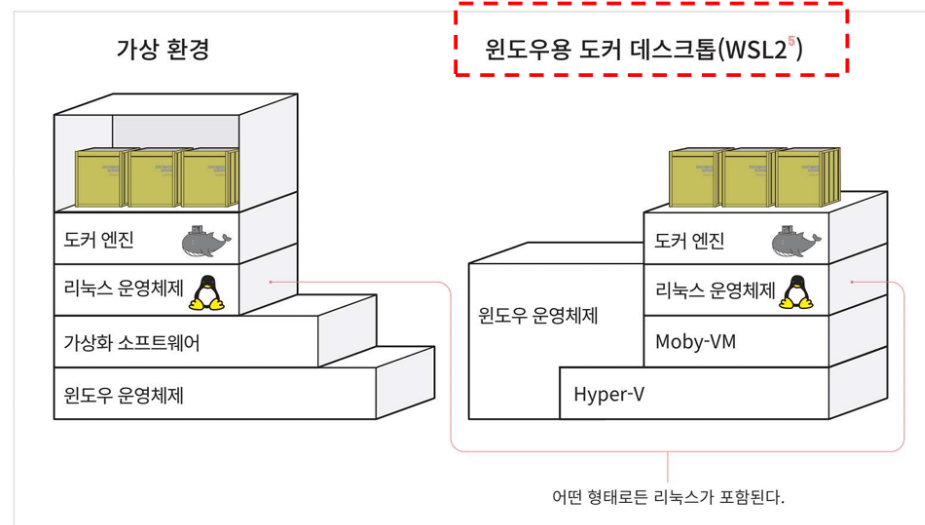
07



도커 기초

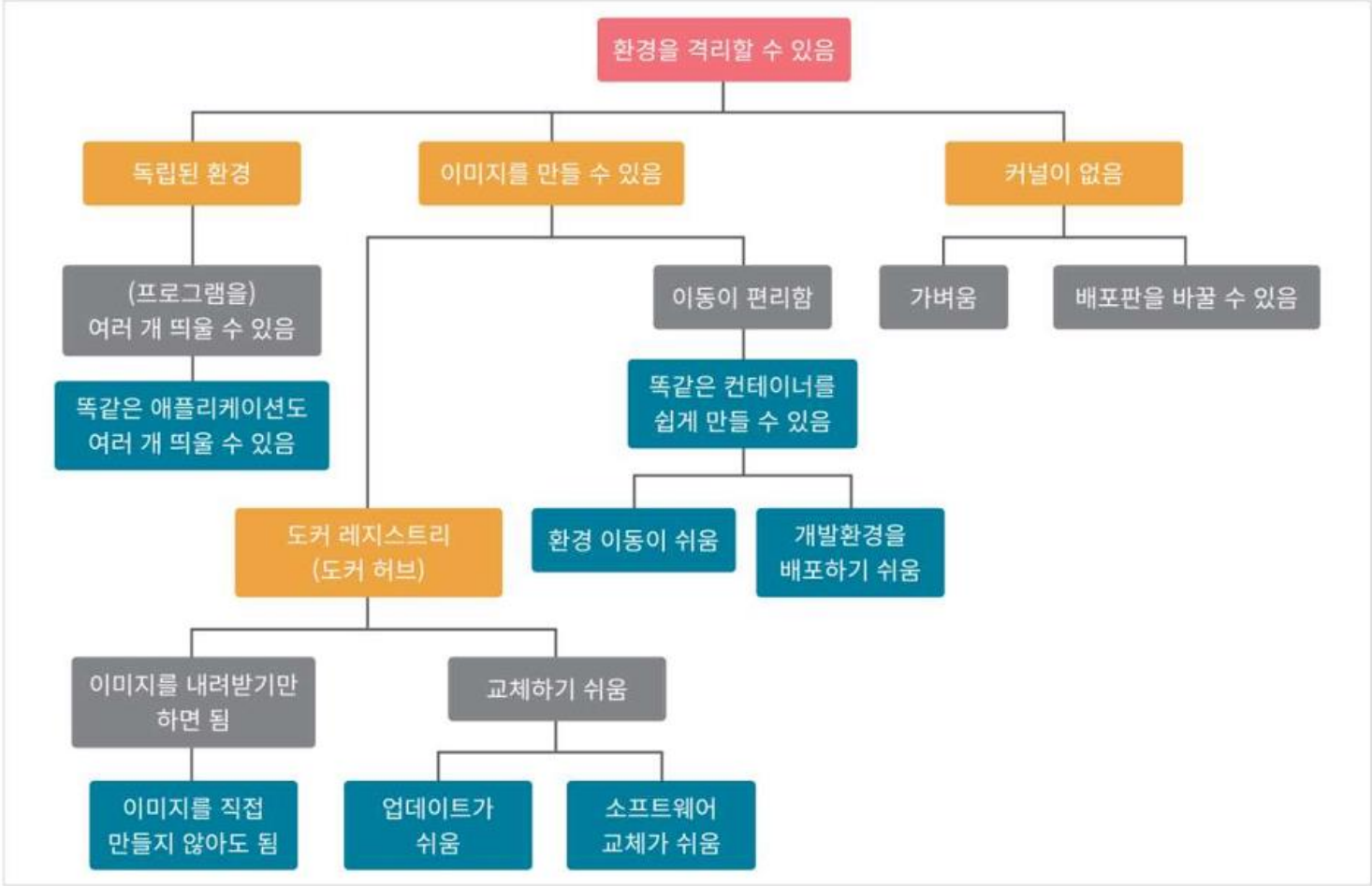
• 리눅스 기반, 그리고 윈도우·macOS에서의 동작

- > 도커는 리눅스 커널 기능(namespace, cgroups)을 기반으로 동작 → 컨테이너 내부 소프트웨어도 리눅스용이어야 함
- > 윈도우·macOS에는 리눅스 커널이 없으므로, Docker Desktop이 경량 가상화 계층 위에 리눅스 환경을 올려 도커 엔진을 구동
 - Windows: Docker Desktop → WSL2 기반 경량 리눅스 VM → 도커 엔진
 - macOS: Docker Desktop → Apple 가상화 프레임워크 기반 경량 리눅스 VM → 도커 엔진
- > 즉, 어떤 운영체제에서 실행하든 내부적으로는 항상 리눅스가 관여함



• 도커의 구조와 특성 및 장단점

> 도커의 특성



• 도커의 구조와 특성 및 장단점

> 장점

- 1대의 물리적인 서버에 여러 대의 서버를 구동할 수 있다.
- 다른 소프트웨어에 영향을 미치지 않아 서버 관리가 용이하다.
- 컨테이너 교체나 수정이 쉬워 환경 이전도 수월하다.
- 명령 한 줄로 서버 구축이 가능하다.

> 단점

- 리눅스 소프트웨어만 지원
- 호스트 서버에 문제가 발생할 시 모든 컨테이너에 영향을 미침

• 도커 활용 주용도

> 팀원에게 동일한 개발환경 제공

- 같은 프로젝트 팀원에게 개발환경을 동일하게 제공
- 프로젝트별 컨테이너를 따로 사용할 수 있음
- 개발환경과 운영환경의 차이가 없음

> 새로운 버전의 테스트

- 새로운 운영체제나 라이브러리 버전을 개발환경에서 먼저 테스트하고 운영환경에 컨테이너 적용가능

> 동일한 서버가 여러 대 필요한 경우

- 1대의 물리 서버에 여러 개의 컨테이너를 이용할 수 있으므로 비용 절약
- 명령 한 줄로 원하는 만큼 서버 구동 가능
- 웹 서버나 API 서버같이 서버를 늘리거나 줄이는데 편해서 많이 사용

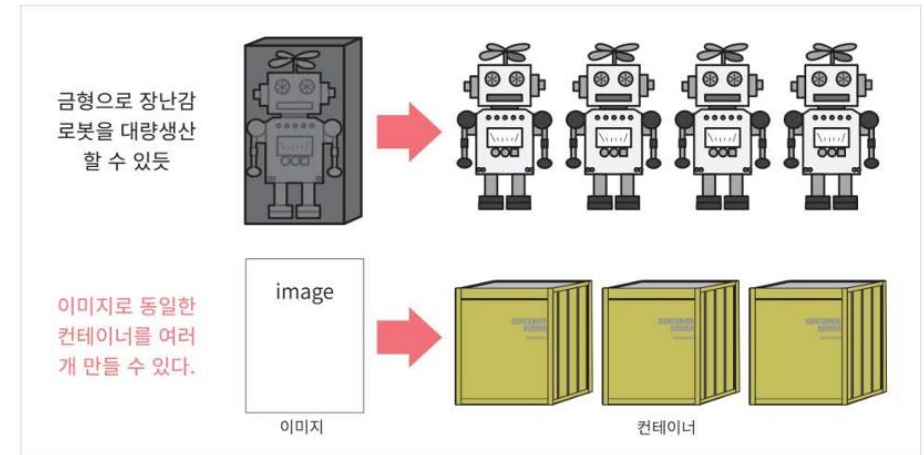
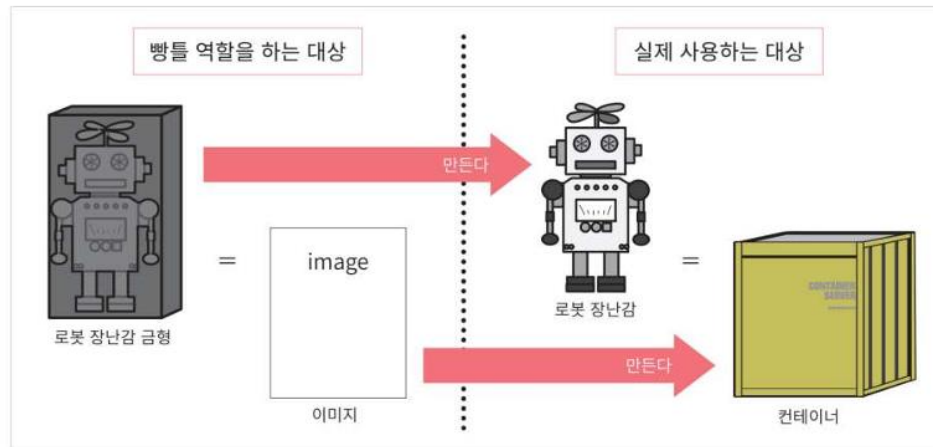
• 이미지와 컨테이너의 관계

> 이미지(Image): 컨테이너를 만들기 위한 “설계도”

- 실행에 필요한 파일·라이브러리·설정을 모두 포함

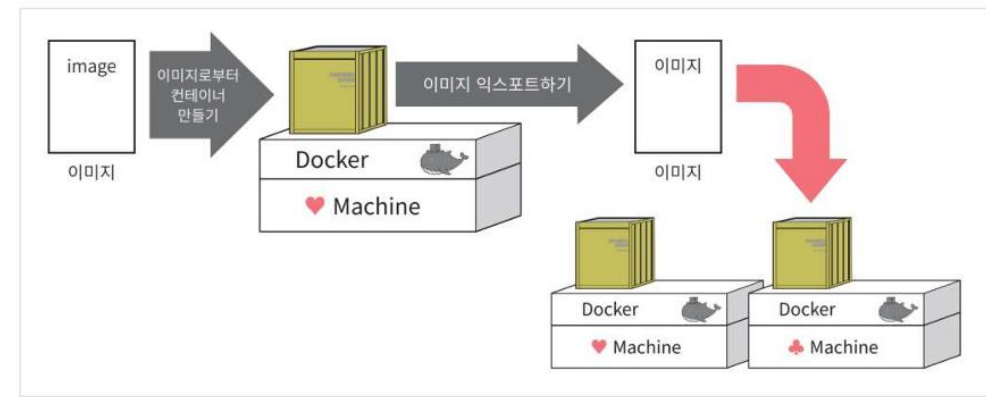
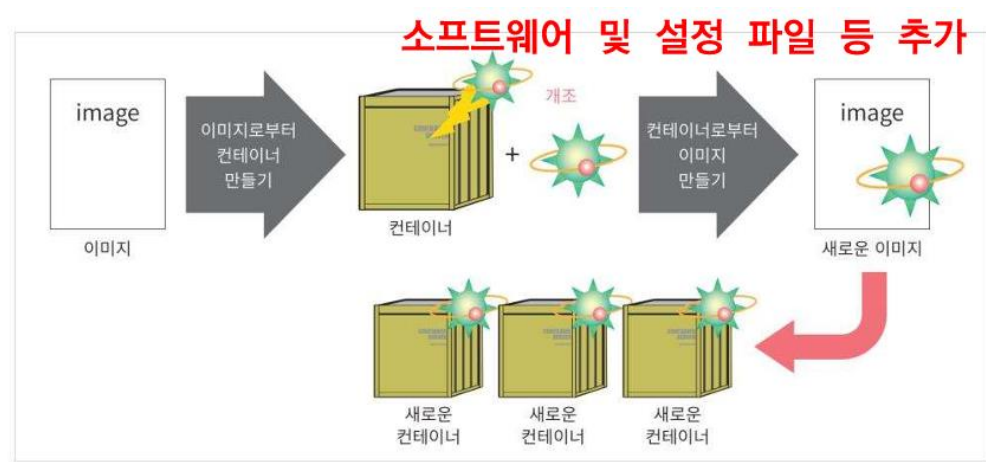
> 컨테이너(Container): 이미지를 바탕으로 실제로 실행되는 인스턴스

> 하나의 이미지로 동일한 컨테이너를 여러 개 생성 가능



• 이미지와 컨테이너의 관계

- > 컨테이너를 수정한 뒤 새로운 이미지로 저장(commit / build)하면,
그 이미지로 또 다른 컨테이너를 만들 수 있음



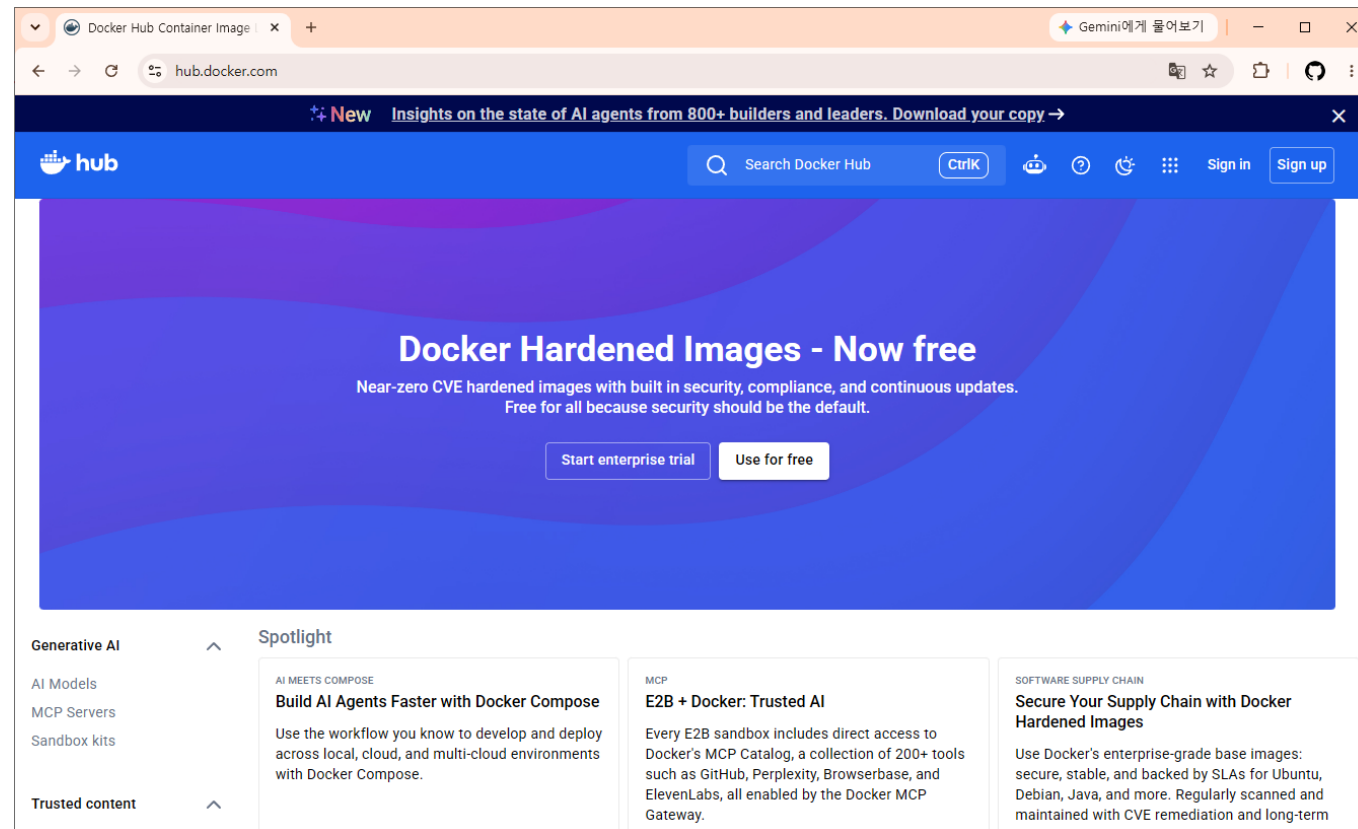
도커 기초

- 도커 허브(Docker Hub)와 이미지

- > 도커 허브(hub.docker.com)

- 도커의 공식 이미지 저장소(레지스트리)

014



• 도커 허브(Docker Hub)와 이미지

> 공식(Official) 이미지 제공

- 같은 소프트웨어라도 OS·버전 조합에 따라 다양한 태그(tag)로 제공

> 안전한 이미지 선택 원칙

- 공식 이미지를 우선 사용
- 필요 시 직접 커스텀 이미지를 만들어 사용

> 설계 원칙

- 한 컨테이너에는 하나의 프로그램만 담는 것을 권장

Machine Learning & AI

IMAGE + 1 MORE



tensorflow/tensorflow
tensorflow

Official Docker images for the machine learning framework TensorFlow (<http://www.tensorflow.org>)

🕒 6h 📥 50M+ ⭐ 2.9K

IMAGE



pytorch/pytorch
pytorch

PyTorch is a deep learning framework that puts Python first.

🕒 15d 📥 10M+ ⭐ 1.7K

IMAGE



langchain/langchain
LangChain

⚡ Building applications with LLMs through composability ⚡

🕒 2y 📥 100K+ ⭐ 388

IMAGE + 1 MORE



python
Docker Official Image

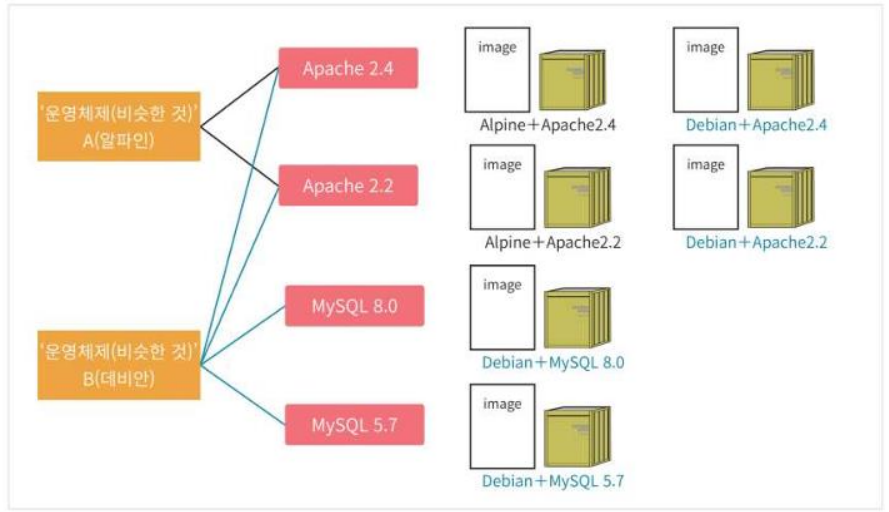
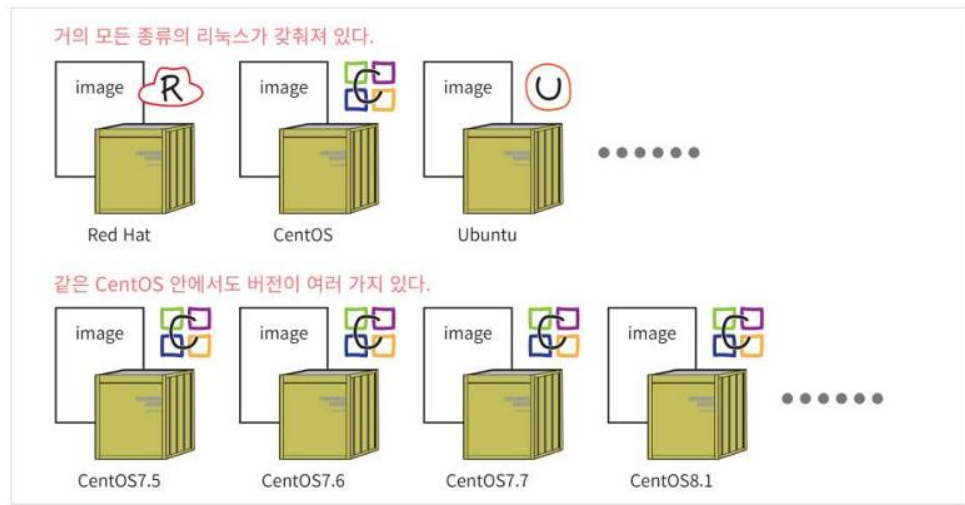
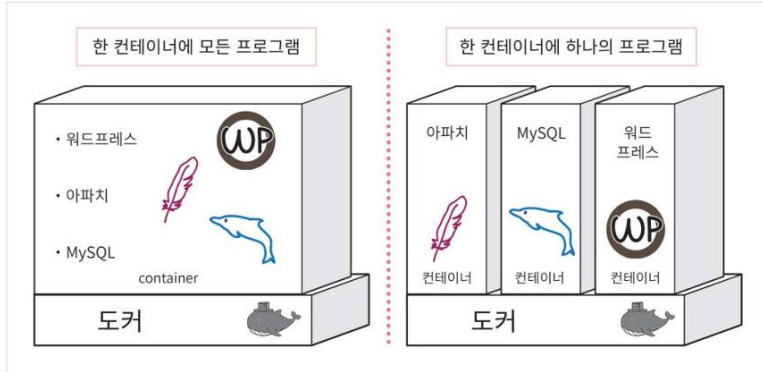
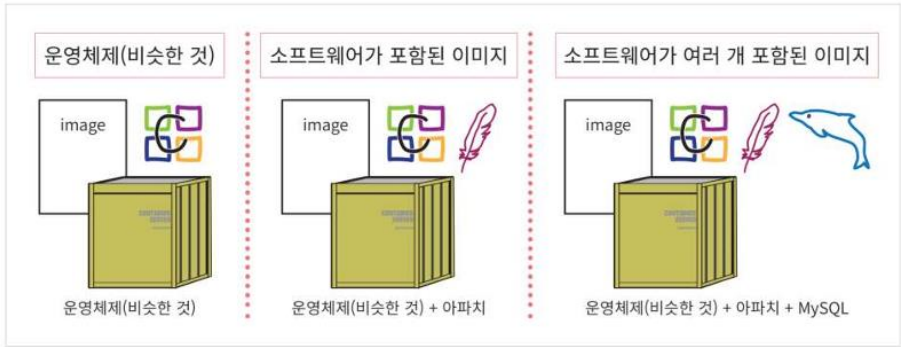
Python is an interpreted, interactive, object-oriented, open-source programming language.

🕒 8d 📥 1B+ ⭐ 10K+

도커 기초

• 도커 허브(Docker Hub)와 이미지

> 도커에서 제공하는 이미지



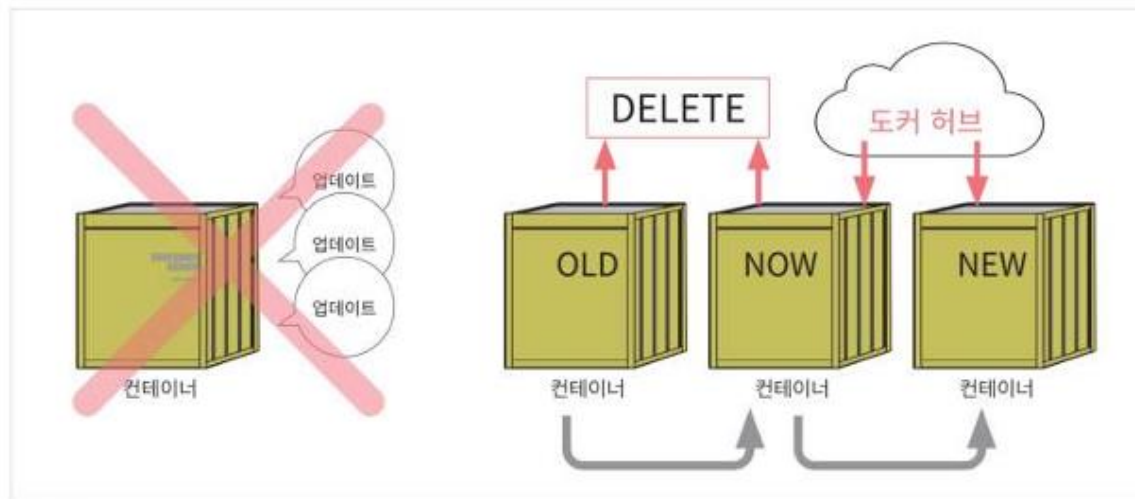
• 컨테이너의 생애주기와 데이터 보존

> 컨테이너는 "일회용" 자원으로 다룸

- 직접 수정하기보다 새 이미지로 새 컨테이너를 만들어 교체

> 생애주기

- 생성(Create) → 실행(Start) → 정지(Stop) → 삭제(Remove)

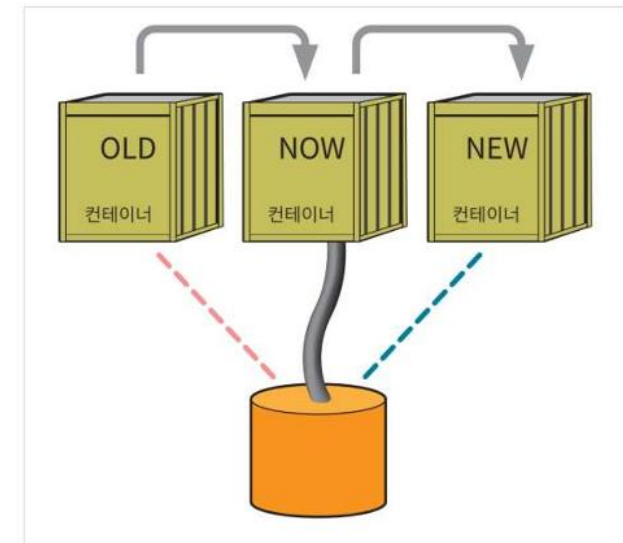
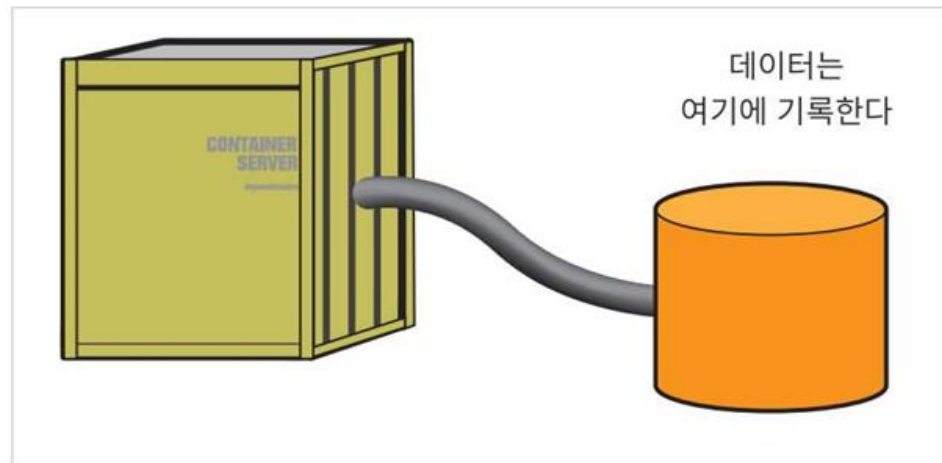


• 컨테이너의 생애주기와 데이터 보존

> 데이터 보존

- 컨테이너를 삭제하면 그 안의 데이터도 함께 사라지므로, 보존이 필요한 데이터는 반드시 볼륨(Volume)으로 컨테이너 외부에 저장
- 외부에 저장된 데이터는 컨테이너가 교체되어도(구버전 → 신버전) 그대로 유지됨

018

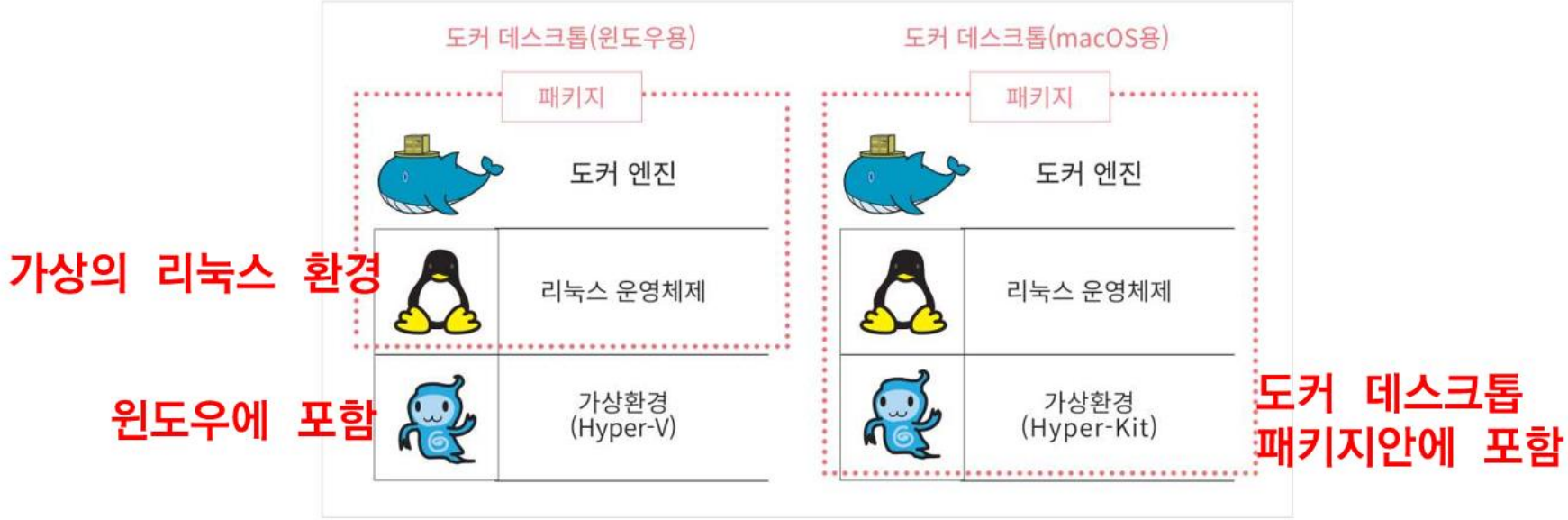


도커 기초

• 도커 실행 환경 준비하기

- > Windows: Windows 11 또는 최신 Windows 10, WSL2 활성화 필요, 64bit
- > macOS: Apple Silicon(M 시리즈) 또는 Intel Mac, 최신 macOS 버전 권장
- > Linux: Ubuntu, Debian, CentOS/RHEL, Fedora 등 주요 배포판 공식 지원
- > 윈도우나 맥은 주로 리눅스 운영체제가 들어있는 도커 데스크톱 이용

019



• 도커 설치하기 (Windows 기준)

- > Windows를 최신 버전으로 업데이트
- > BIOS 가상화 활성화
- > PowerShell(관리자)에서 `wsl --install` 실행
- > **재부팅**
- > Docker Desktop 설치

> macOS·Linux는 각각 공식 설치 가이드(docs.docker.com)를 따라 설치

• 도커 엔진 시작과 종료

- > 도커 엔진이 꺼지면 실행 중이던 컨테이너도 모두 함께 정지됨
- > 엔진은 자동 실행되어도 컨테이너는 자동으로 실행되지 않음
 - 필요 시 --restart 옵션으로 지정

OS	시작	종료	자동 실행 설정
윈도우	시작 메뉴 → Docker Desktop 실행	Docker Desktop 창 닫기	기본적으로 로그인 시 자동 실행
macOS	Launchpad → Docker Desktop 실행	메뉴 막대 → Quit Docker Desktop	Docker Desktop 설정에서 지정
리눅스	sudo systemctl start docker	sudo systemctl stop docker	sudo systemctl enable docker

• 도커 명령어의 기본 구조

- > 상위명령: container, image, volume, network 등 다룰 자원의 종류
- > 하위명령: run, start, stop, rm, pull, ls 등 수행할 동작
- > start·stop·run처럼 자주 쓰는 컨테이너 명령은 상위명령(container) 생략 가능
- > 옵션으로 세부 동작 지정: -d(백그라운드) · -p(포트) · -v(볼륨) · -e(환경변수)

docker [상위명령] [하위명령] [옵션] [대상] [인자]

예) docker container run -d --name web -p 8080:80 httpd

• 자주 쓰는 도커 명령어 cheatsheet

구분	명령	설명
컨테이너	docker run	이미지를 받아 컨테이너 생성 + 실행
컨테이너	docker ps [-a]	실행 중(전체) 컨테이너 목록 확인
컨테이너	docker stop / start	컨테이너 정지 / 재시작
컨테이너	docker rm	정지된 컨테이너 삭제
컨테이너	docker exec	실행 중인 컨테이너에 명령 전달
이미지	docker pull / image ls	이미지 내려받기 / 목록 확인
이미지	docker build	Dockerfile로 이미지 생성
이미지	docker image rm	이미지 삭제
볼륨/네트워크	docker volume create/ls/rm	볼륨 생성 · 조회 · 삭제
볼륨/네트워크	docker network create/ls/rm	네트워크 생성 · 조회 · 삭제

- 도커 명령어
- 컨테이너 제어 명령

docker [container] 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
start	컨테이너를 실행	O	-i
stop	컨테이너를 정지	O	거의 사용하지 않음
create	도커 이미지로부터 컨테이너를 생성	O	--name -e -p -v
run	도커 이미지를 내려받고 컨테이너를 생성해 실행함(다운 로드는 필요한 경우에만). docker image pull, docker container create, docker container start라는 세 개의 명령을 하나로 합친 것과 같다.	O	--name -e -p -v -d -i -t
rm	정지 상태의 컨테이너를 삭제	O	-f -v
exec	실행 중인 컨테이너 속에서 프로그램을 실행	O	-i -t
ls	컨테이너 목록을 출력	*1	-a
cp	도커 컨테이너와 도커 호스트 간에 파일을 복사	O	거의 사용하지 않음
commit	도커 컨테이너를 이미지로 변환	O	거의 사용하지 않음

*1: 생략형은 docker ps

- 도커 명령어
- 이미지 제어 명령

`docker image` 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
pull	도커 허브 등의 리포지토리에서 이미지를 내려받음	O	거의 사용하지 않음
rm	도커 이미지를 삭제	*2	거의 사용하지 않음
ls	내려 받은 이미지의 목록을 출력	X	거의 사용하지 않음
build	도커 이미지를 생성	O	-t

*2: 생략형은 `docker rmi`

- 도커 명령어
- 볼륨 제어 명령

docker volume

하위명령

옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
create	볼륨을 생성	X	--name
inspect	볼륨의 상세 정보를 출력	X	거의 사용하지 않음
ls	볼륨의 목록을 출력	X	-a
prune	현재 마운트되지 않은 볼륨을 모두 삭제	X	거의 사용하지 않음
rm	지정한 볼륨을 삭제	X	거의 사용하지 않음

- 도커 명령어
- 네트워크 제어 명령

docker network 하위명령어 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
connect	컨테이너를 도커 네트워크에 연결	X	거의 사용하지 않음
disconnect	컨테이너의 도커 네트워크 연결을 해제	X	거의 사용하지 않음
create	도커 네트워크를 생성	X	거의 사용하지 않음
inspect	도커 네트워크의 상세 정보를 출력	X	거의 사용하지 않음
ls	도커 네트워크의 목록을 출력	X	거의 사용하지 않음
prune	현재 컨테이너가 접속하지 않은 네트워크를 모두 삭제	X	거의 사용하지 않음
rm	지정한 네트워크를 삭제	X	거의 사용하지 않음

- 도커 명령어
- 기타 명령어

그 밖의 상위 커맨드

상위 커맨드	내용
checkpoint	현재 상태를 일시적으로 저장한 후, 나중에 해당 시점의 상태로 되돌릴 수 있다. 현재 실험적 기능이다.
node	도커 스웜의 노드를 관리하는 기능
plugin	플러그인을 관리하는 기능
secret	도커 스웜의 비밀값 정보를 관리하는 기능
service	도커 스웜의 서비스를 관리하는 기능
stack	도커 스웜 또는 쿠버네티스에서 여러 개의 서비스를 합쳐 구성한 스택을 관리하는 기능
swarm	도커 스웜을 관리하는 기능
system	도커 엔진의 정보를 확인하는 기능

- 단독 명령어

단독 커맨드⁹

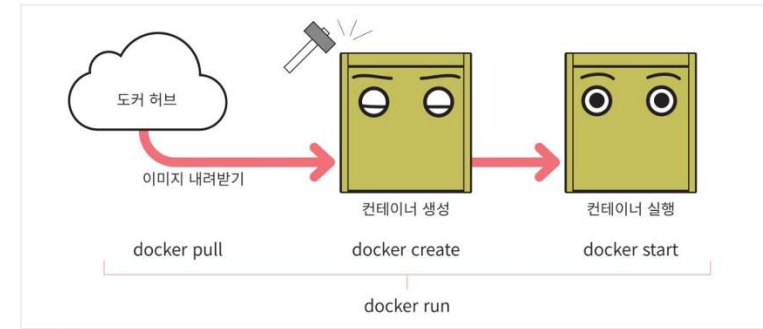
단독 커맨드	내용	주요 옵션
login	도커 레지스트리에 로그인	-u -p
logout	도커 레지스트리에 로그아웃	거의 사용하지 않음
search	도커 레지스트리를 검색	거의 사용하지 않음
version	도커 엔진 및 명령행 도구의 버전을 출력	거의 사용하지 않음

도커 기초

• 도커 생성 및 실행

> docker container run

- 도커 생성 및 실행 기능, 필요시 이미지 다운로드도 실행
- docker run = docker pull + create + start



> 컨테이너 생성 실습

```
docker run -dit --name my-ubuntu ubuntu:latest
```

> 생성 확인 docker ps

```
docker ps
```

```
C:\Users\admin>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
67892bf98ef5   ubuntu:latest  "/bin/bash"             3 minutes ago Up 3 minutes   my-ubuntu
```

> 종료 및 삭제

```
docker stop my-ubuntu
```

```
docker rm my-ubuntu
```

- 도커 생성 및 실행
- 컨테이너 생성 및 실행

```
docker run 옵션 이미지 [인자]
```

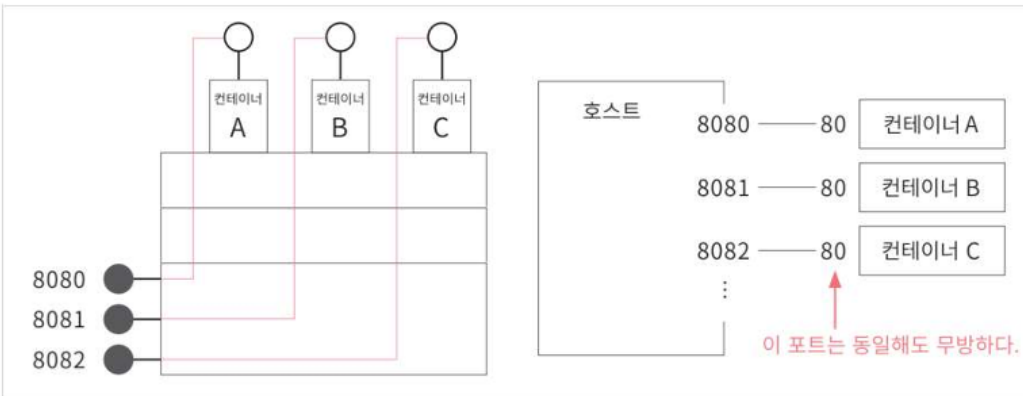
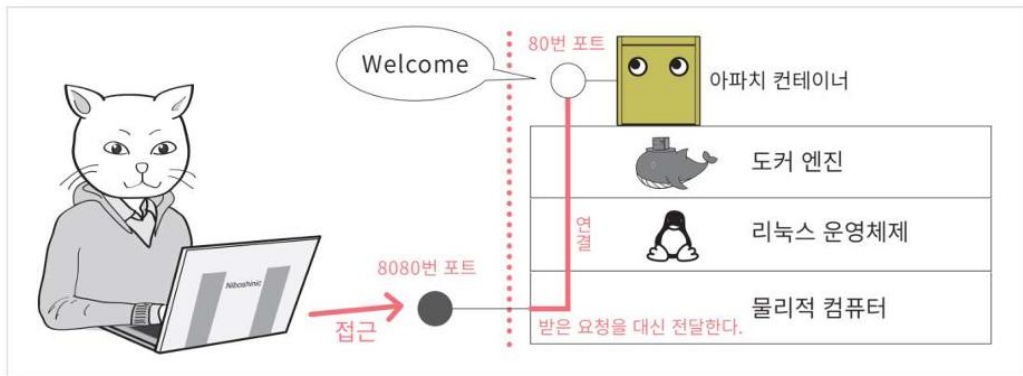
주요 옵션

옵션 형식	내용
--name 컨테이너_이름	컨테이너 이름을 지정함
-p 호스트_포트번호:컨테이너_포트번호	포트 번호를 지정함
-v 호스트_디스크:컨테이너_디렉터리	볼륨을 마운트함
--net=네트워크_이름	컨테이너를 네트워크에 연결함
-e 환경변수_이름=값	환경변수를 설정함
-d	백그라운드로 실행함
-i	컨테이너와 대화형 모드로 연결함
-t	컨테이너에 터미널 연결함
-help	사용 방법 안내 메시지를 출력함

-p는 --publish, -v는 --volume, -e는 --env, -d는 --detach, -i는 --interactive, -t는 --tty의 생략형이다.

• 컨테이너와 통신하기 — 포트 포워딩

- > 컨테이너는 기본적으로 외부에서 접근X, 실행 시점에 포트를 연결해야 통신이 가능함
- > -p 호스트포트:컨테이너포트 형식으로 지정
 - 예: -p 8080:80 → 호스트 8080번 접속을 컨테이너 80번으로 전달
- > 여러 컨테이너를 동시에 띄울 때는 호스트 포트가 겹치지 않게 지정

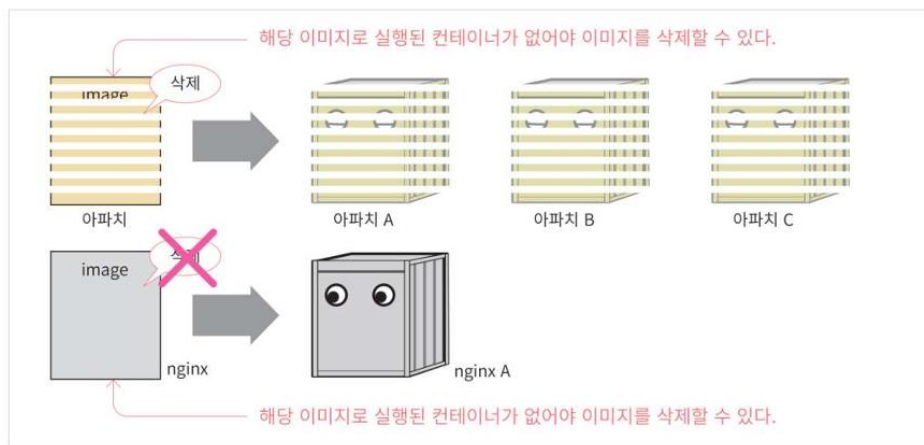
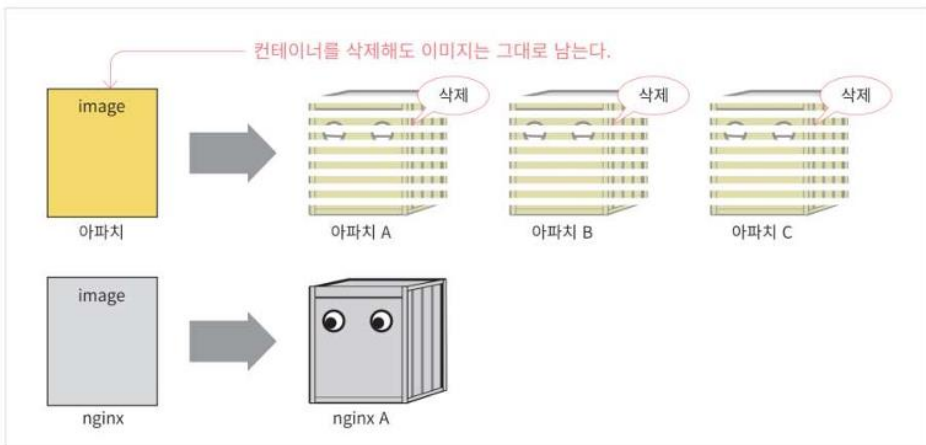


• 이미지 삭제 규칙

- > 컨테이너를 삭제해도 이미지는 그대로 남아있으므로, 쓰지 않는 이미지는 별도로 삭제해야 함
- > 단, 그 이미지로 만든 컨테이너가 하나라도 남아있으면 이미지를 삭제할 수 없음
- > 태그를 지정하지 않으면 자동으로 latest 태그가 사용됨
 - 실무에서는 재현성을 위해 버전 태그를 명시하는 것을 권장

032

```
docker image ls  
docker image rm 이미지명[:태그]
```



• 실습 - 컨테이너 만들기

> Mysql

- mysql://root:1234@localhost:13306/mydb

```
docker run -d --name my-mysql -e MYSQL_ROOT_PASSWORD=1234 -e  
MYSQL_DATABASE=mydb -p 13306:3306 mysql:9
```

> Node.js

- server.js 파일을 현재 경로에 준비

```
docker run -d --name node-web -p 8080:3000 node:alpine sh -c "mkdir -p /app &&  
while [ ! -f /app/server.js ]; do sleep 1; done; exec node /app/server.js"
```

```
docker cp ./server.js node-web:/app/server.js
```

- 호스트와 컨테이너 간 파일 복사

- > `docker cp` 호스트경로 컨테이너이름:컨테이너경로

- 호스트에서 컨테이너로 복사

- > `docker cp` 컨테이너이름:컨테이너경로 호스트경로

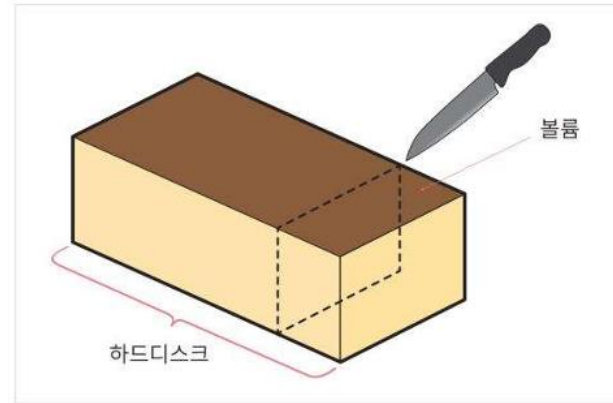
- 반대 방향(컨테이너 → 호스트)도 동일하게 가능

```
docker cp ./server.js node-web:/app/server.js  
docker cp node-web:/app/server.js ./server.js
```

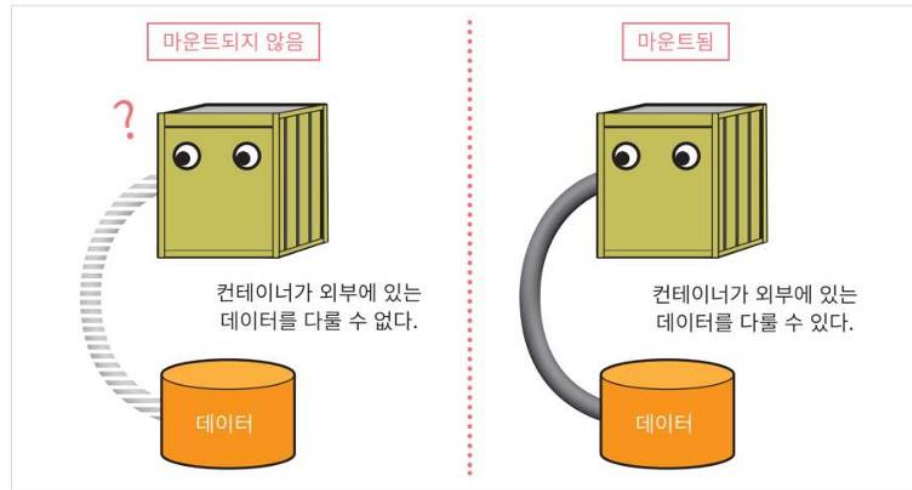
• 볼륨과 마운트

> 컨테이너는 삭제해도 데이터는 남아야 하므로 반드시 외부 저장소를 마운트해서 사용

> 볼륨(Volume) = 저장소



> 마운트(Mount) = 연결



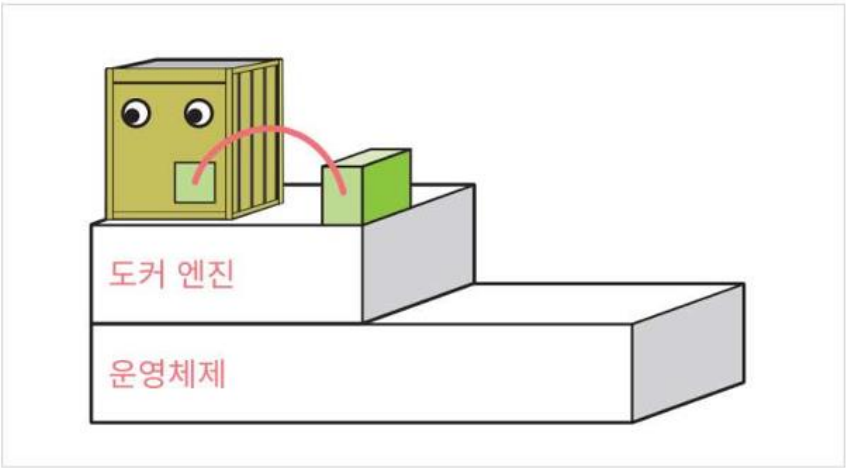
• 볼륨 마운트 vs 바인드 마운트

> 바인드 마운트 (-v 호스트경로:컨테이너경로)

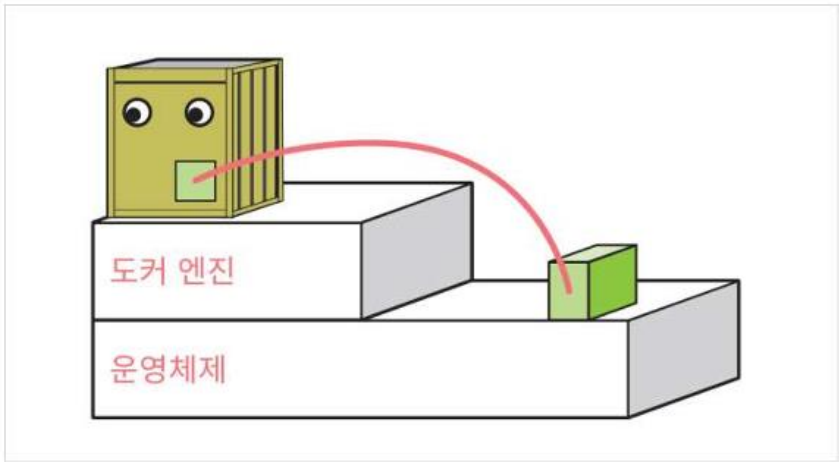
> 볼륨 마운트 (-v 볼륨이름:컨테이너경로)

항목	볼륨 마운트	바인드 마운트
스토리지 위치	도커 엔진 관리 영역	호스트의 임의 디렉터리
편집 방법	컨테이너를 통해서만	호스트에서 직접 편집 가능
권장 용도	도커 공식 권장, 백업·이관	자주 수정하는 개발용 파일에 적합

<볼륨 마운트>



<바인드 마운트>



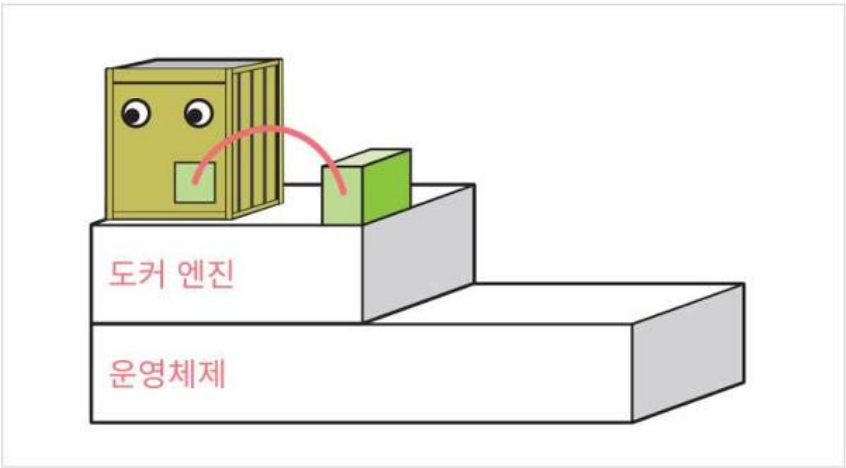
• 볼륨 마운트 vs 바인드 마운트

> 바인드 마운트 (-v 호스트경로:컨테이너경로)

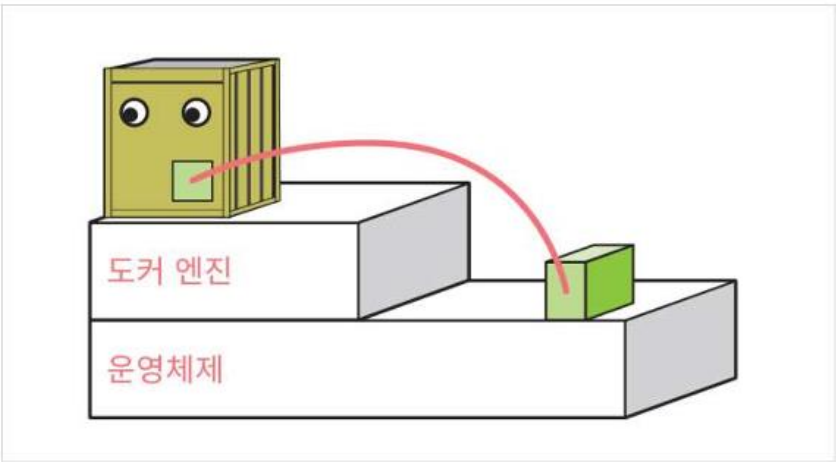
> 볼륨 마운트 (-v 볼륨이름:컨테이너경로)

항목	볼륨 마운트	바인드 마운트
스토리지 위치	도커 엔진 관리 영역	호스트의 임의 디렉터리
편집 방법	컨테이너를 통해서만	호스트에서 직접 편집 가능
권장 용도	도커 공식 권장, 백업·이관	자주 수정하는 개발용 파일에 적합

<볼륨 마운트>



<바인드 마운트>



• 실습

볼륨 마운트

```
docker run --rm -it --name volume-practice -v practice-data:/data ubuntu:latest bash
```

- practice-data 라는 이름의 볼륨이 생성(기존에 있으면 그대로 연결)

바인드 마운트

```
docker run --rm -it --name bind-practice -v "${PWD}:/data" ubuntu:latest bash
```

\${PWD}는 현재 경로를 의미(현재 경로와 연결)

• 나만의 이미지 만들기 — 두 가지 방법

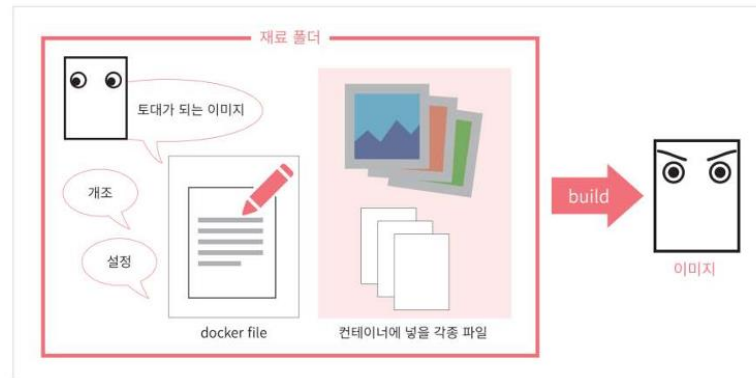
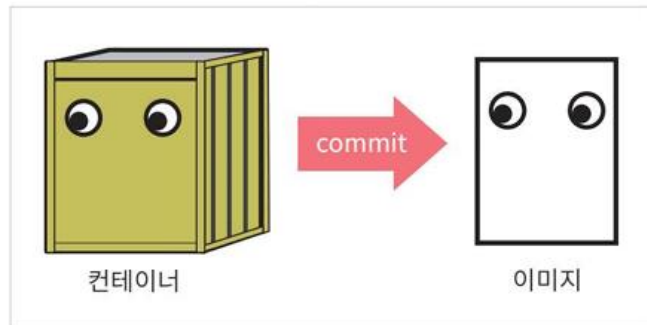
> docker commit

- 실행 중이거나 수정한 컨테이너를 그대로 이미지로 굳히는 방법
- 빠르지만 재현성이 낮음

> Dockerfile

- “기반 이미지 + 파일 추가 + 실행 명령”을 스크립트로 작성해 docker build로 이미지 생성
- 재현 가능, 실무 권장

docker commit 컨테이너이름 새이미지이름
docker build -t 이미지이름 .



• Dockerfile 주요 인스트럭션

인스트럭션	내용
FROM	토대가 되는 이미지를 지정
COPY / ADD	이미지에 파일이나 폴더를 추가
RUN	이미지를 빌드할 때 실행할 명령어를 지정
CMD / ENTRYPOINT	컨테이너를 실행할 때 실행할 명령어를 지정
WORKDIR	명령어를 실행하는 작업 디렉터리를 지정
ENV / ARG	환경변수 정의 / 빌드 시 입력받을 인자 선언
EXPOSE	컨테이너가 통신에 사용할 포트를 명시
HEALTHCHECK	컨테이너 상태 확인 방법을 커스터마이징

• 실습 — Dockerfile로 이미지 빌드하기

> Dockerfile

```
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \#
    python3 \#
    python3-pip \#
    python3-venv \#
    openssh-server \#
    ca-certificates \#
    curl \#
    git \#
    && rm -rf /var/lib/apt/lists/*

RUN useradd -m -s /bin/bash dev \#
    && mkdir -p /run/sshd /home/dev/workspace \#
    && chown -R dev:dev /home/dev/workspace

RUN printf 'PasswordAuthentication yes\nPermitRootLogin no\n' \#
    > /etc/ssh/sshd_config.d/00-dev.conf

WORKDIR /home/dev/workspace

EXPOSE 22

CMD ["/bin/sh", "-c", "ssh-keygen -A && exec /usr/sbin/sshd -D -e"]
```

```
docker build -t python-ssh .
```

• 이미지·컨테이너를 파일로 옮기기

> save / load는 이미지의 레이어·히스토리를 모두 보존

> export / import는 파일시스템만 보존(더 가벼움)

```
docker save -o myimage.tar 이미지이름      # 이미지 → tar 파일
docker load -i myimage.tar                  # tar 파일 → 이미지

docker export 컨테이너이름 > container.tar  # 컨테이너 파일시스템 → tar
docker import container.tar 새이미지이름    # tar → 이미지
```

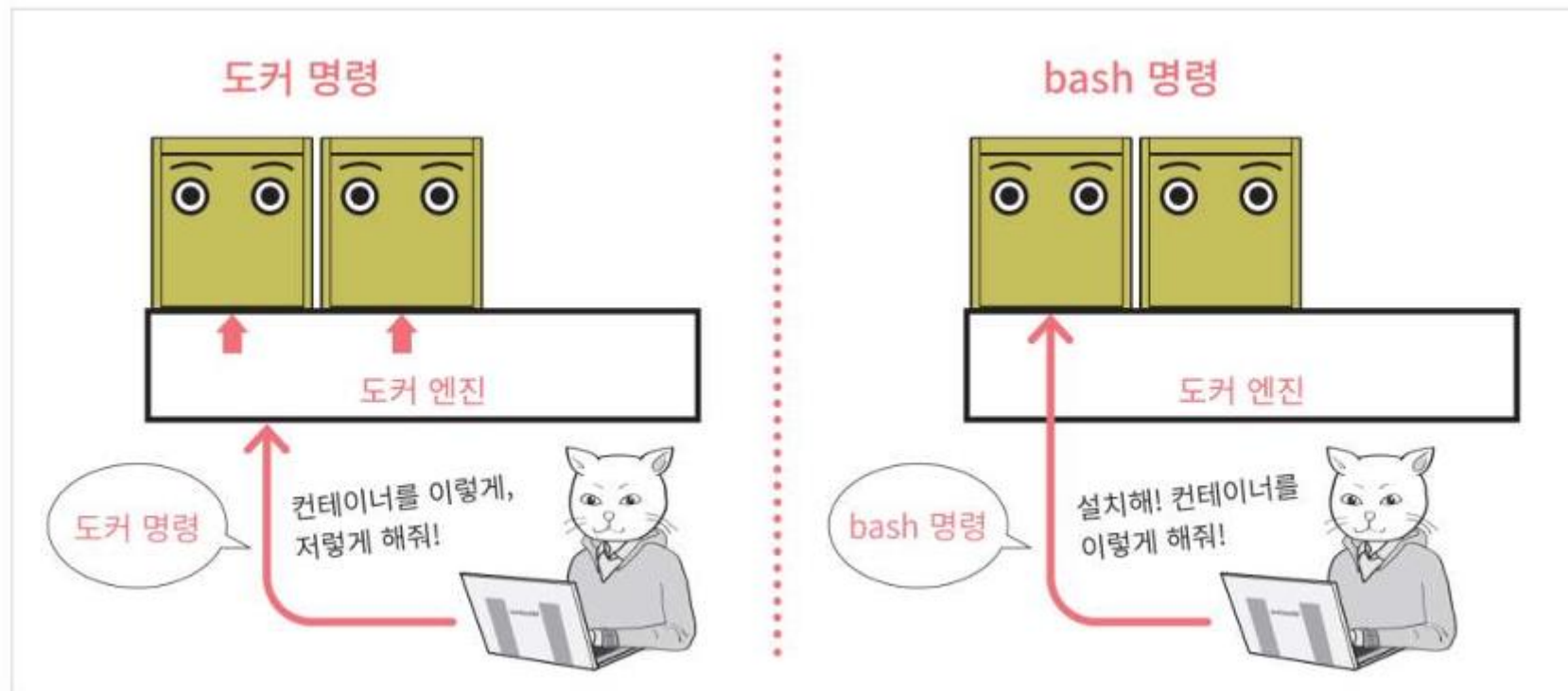
도커 기초

• 컨테이너 내부에 들어가서 개조하기

> docker exec: 실행 중인 컨테이너에 명령 전달

> 컨테이너 내부 bash에 들어가 있는 동안에는 도커 명령을 사용할 수 없음

```
docker exec -it my-ubuntu /bin/bash
```



• SSH 연결하기

> SSH(SecureShell)

- 네트워크 상의 다른 컴퓨터에 로그인하거나 원격 시스템에서 명령을 실행하고 다른 시스템으로 파일을 복사할 수 있도록 해 주는 응용 프로그램 또는 그 프로토콜

파이썬 컨테이너 생성

```
docker run -d --name python-dev -p 127.0.0.1:22222:22 -v python-workspace:/home/dev/workspace python-ssh
```

dev 계정 비밀번호 등록

```
docker exec -it python-dev passwd dev
```

컨테이너 내부 작업공간

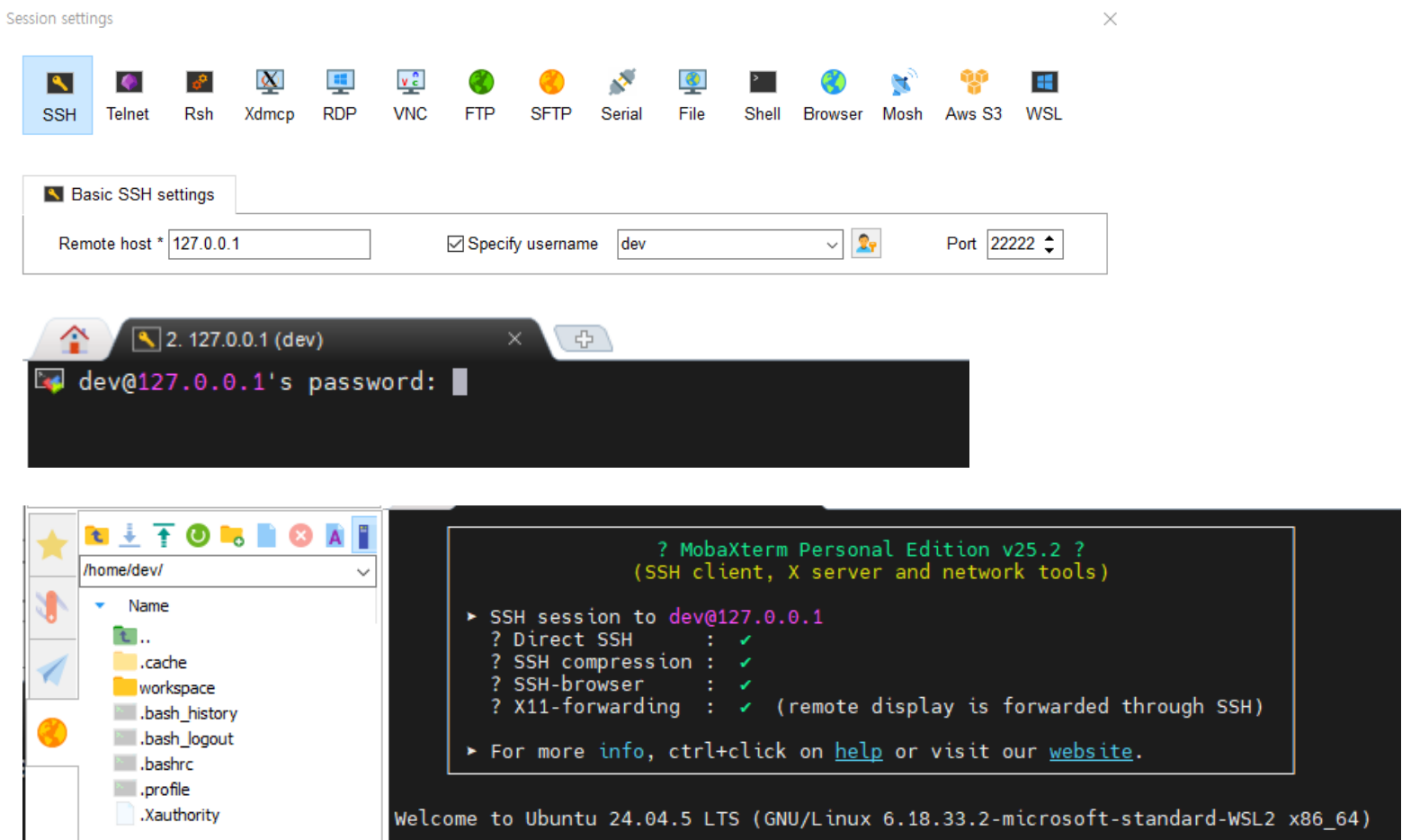
```
docker exec python-dev chown dev:dev /home/dev/workspace
```

#ssh 접속확인

```
ssh -p 22222 dev@localhost
```

• SSH 연결하기

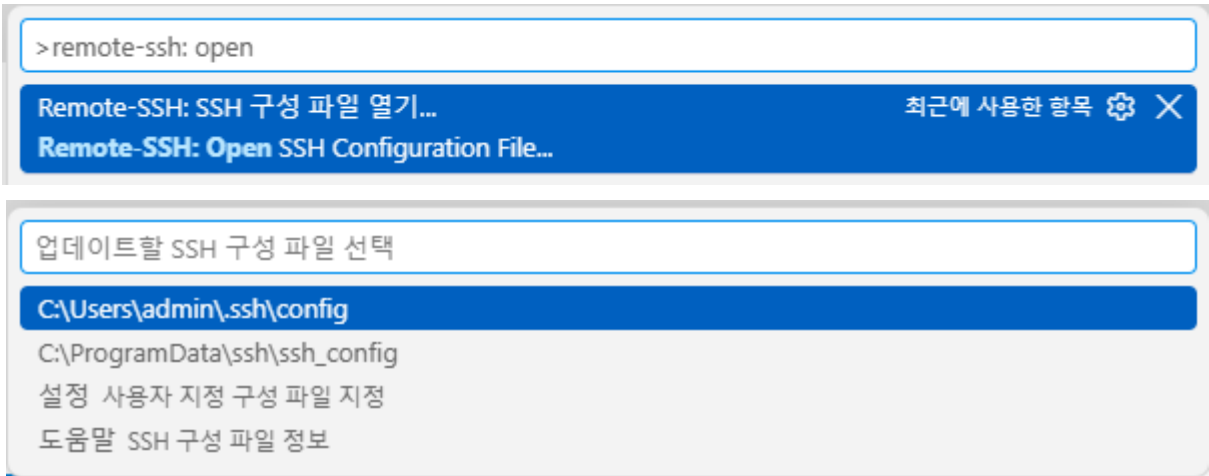
> MobaXterm으로 연결



• SSH 연결하기

> VS code로 연결

- ssh/config 파일 수정



```
1 # Read more about SSH co
2 Host python-docker
3     HostName 127.0.0.1
4     User dev
5     Port 22222
```

• SSH 연결하기

> VS code로 연결

- ssh 원격 연결

